ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF COMPUTER AND
INFORMATICS

TinyNPU

Graduation Project Final Report

Fırat Kızılboğa
150200115

Department: Computer Engineering
Division: Computer Engineering

Advisor: Assoc. Prof. Dr. Ayse Yilmazer

March 2026

Statement of Authenticity

I/we hereby declare that in this study

1. all the content influenced from external references are cited clearly and in detail,
2. and all the remaining sections, especially the theoretical studies and implemented software/hardware that constitute the fundamental essence of this study is originated by my/our individual authenticity.

İstanbul, Mart 2026

Fırat Kızılboğa

Acknowledgments

I would like to thank my advisor, Assoc. Prof. Dr. Ayse Yilmazer, for her guidance throughout this project. I also thank the people whose open-source tooling and research made it possible to build, analyze, and validate the TinyNPU hardware and software stack.

TinyNPU

(SUMMARY)

TinyNPU is a compact accelerator project that evolved into a complete hardware/-software stack. The project now includes a parameterized systolic-array-based neural processing unit, a PyTorch-first segmented compiler and runtime, a quantization-aware model preparation flow, RTL and host-side verification infrastructure, and performance and synthesis characterization tooling. The goal of the project is not merely to design a datapath, but to make that datapath usable from a realistic machine-learning workflow and measurable in a disciplined way.

The hardware architecture is organized around an $N \times N$ output-stationary systolic array, a unified buffer, an instruction memory, streamer logic that feeds the array, and a post-processing unit that applies bias, rescaling, activation, and packed writeback. The software stack traces PyTorch models, partitions them into accelerator segments and host operations, compiles supported segments into TinyNPU instruction and buffer images, and executes the resulting plan on either host emulation or RTL simulation. Unsupported work is never hidden; it appears explicitly as host-side steps in the execution plan.

One of the main lessons of the project is that the useful unit of design is the whole stack, not the RTL alone. Compiler-visible tensor semantics, physical packing format, host/NPU boundaries, verification points, and repeated-run runtime behavior all affect whether the accelerator is practical to use. For this reason, the project includes explicit host-op dispatch, compiler-owned expected tensors for verification, repeated-run sessions with persistent state, and benchmark accounting that separates accelerator compute cycles from runtime and protocol overhead.

The current experimental evidence is intentionally narrow but concrete. The implemented stack has been validated end-to-end on a trained MNIST model, and a controlled multi-tile matrix multiplication benchmark has been used to study compute speedup and integration overhead. In addition, a reproducible Yosys-based synthesis flow provides rough resource estimates for the current design. The report therefore presents TinyNPU not as a finished industrial inference platform, but as a working accelerator research system whose main value lies in understanding architecture, compilation, packing, and workload fit together.

TinyNPU

(ÖZET)

TinyNPU, özel bir nöral işlemci birimi ile onu kullanılabilir hale getiren derleyici, çalışma zamanı, niceleme akışı ve benzetim tabanlı doğrulama ortamını birlikte geliştirmeyi amaçlayan bir projedir. Projenin hedefi yalnızca küçük bir hızlandırıcı çekirdeği tasarlamak değil, bu çekirdeği gerçek bir makine öğrenmesi iş akışında kullanılabilir hale getirmektir. Bu nedenle proje, PyTorch tabanlı model hazırlama sürecini destekleyen, nicelemeli modelleri hızlandırıcı segmentlerine dönüştüren, desteklenmeyen işlemleri ana işlemci üzerinde çalıştıran ve sonuçları yazılım referanslarıyla doğrulayan bütünleşik bir donanım-yazılım yapısı içermektedir.

Donanım tarafında TinyNPU, birleşik tampon bellek ve komut belleği ile beslenen sistolik dizi tabanlı bir hesaplama çekirdeği etrafında kurulmuştur. Girdi aktivasyonları ve ağırlıklar birleşik tampondan akıtıcı birimlere, oradan da sistolik diziye iletilir. Hesaplama sonucunda oluşan değerler son işleme biriminden geçirilerek paketlenmiş biçimde tekrar belleğe yazılır. Bu yapı, yoğun nicelemeli matris işlemleri için verimli bir veri yolu sağlarken, dışarıdan görülen programlama modelini de yönetilebilir düzeyde tutmaktadır. Yazılım tarafı ise yüksek seviyeli model işlemlerini hızlandırıcı segmentleri ve gerekirse ana işlemci işlemlerini birlikte içeren bir yürütme planına dönüştürmektedir.

Bu projedeki temel ilerleme, izole donanım deneylerinden uçtan uca çalışan bir sisteme geçilmesi olmuştur. Eğitilmiş MNIST modeli dahil gerçek iş yükleri yazılım ve donanım yığını üzerinden çalıştırılabilmektedir. Ayrıca, hızlandırıcının gerçek hesaplama çevrimlerini entegrasyon ek yüklerinden ayıran bir ölçüm altyapısı geliştirilmiştir. Böylece çekirdeğin kendi verimliliği ile yazılım, paketleme ve arayüz maliyetleri ayrı ayrı değerlendirilebilmek Tekrarlı çıkarım senaryolarında gereksiz durum yüklemelerini azaltmak için oturum tabanlı çalışma ve ağırlık kalıcılığı desteği de eklenmiştir.

Projenin mevcut durumu, donanımın anlamlı hesaplama yaptığı, ancak sistem seviyesindeki davranışın derleyici bölütlemesi, bellek yerleşimi ve çalışma zamanı düzenlemesine güçlü biçimde bağlı olduğunu göstermektedir. Bu nedenle proje, "derleyiciyi var etmek" aşamasını aşmış; bunun yerine derleyici ve çalışma zamanının karmaşıklığını yönetmek, gereksiz çevrim maliyetlerini azaltmak ve hangi iş yüklerinin bu mimariye uygun olduğunu anlamak gibi daha olgun tasarım sorularına yönelmiştir. Bu rapor, ortaya çıkan sistemi, temel tasarım kararlarını ve işlevsellik ile performans değerlendirmesinde kullanılan yöntemi özetlemektedir.

Contents

1 Introduction and Project Summary **1**
1.1 Background and Motivation 1
1.2 Problem Definition 1
1.3 Project Scope and Approach 1

2 Comparative Literature Survey **3**
2.1 Mixed-Precision Quantization 3
2.2 High-Performance Matrix Multiplication 3
2.3 Existing Accelerator Architectures 3

3 Developed Approach and System Model **5**
3.1 System Overview 5
3.2 NPU 5
3.3 Quantizer and Model Preparation 8
3.4 Compiler 9
3.5 Runtime and Verification 11
3.6 Current Scope Boundaries 13

4 Experimental Setup and Design **14**

5 Comparative Evaluation and Discussion **15**

6 Conclusion and Future Work **17**

A Benchmark Cost Model **17**

1 Introduction and Project Summary

1.1 Background and Motivation

Modern deep neural networks demand substantial compute and memory bandwidth even at inference time. This has made specialized accelerator design a central topic both for datacenter-scale systems and for smaller edge-oriented deployments [6, 4]. Quantization is one of the main techniques that makes such deployment practical, because it reduces memory traffic and allows arithmetic throughput to increase when narrow integer types are used instead of floating-point data.

However, quantization is not a single fixed target. Mixed-precision work has shown that different layers or kernels inside the same model can tolerate different bit-widths [7]. This creates a hardware opportunity and a software problem at the same time. Hardware should benefit from narrower arithmetic when it is safe to do so, but the software stack must still prepare the model, preserve numerical intent, and decide where accelerator execution ends and host execution begins.

1.2 Problem Definition

The main problem addressed by TinyNPU is therefore not only how to build a small systolic-array accelerator, but how to make mixed-precision quantized computation usable from a realistic model flow. Fixed-precision accelerators leave throughput on the table when different parts of a model have different precision needs. At the other end of the spectrum, CPU-centric approaches based on ISA extensions or soft SIMD can support low-bit computation, but they still pay substantial software-side cost for packing, unpacking, and orchestrating the data movement around those operations [3, 1, 2].

This project treats the problem as a co-design task. The hardware must expose a regular datapath that benefits from packed low-precision arithmetic. The compiler and runtime must make tensor layout, quantization boundaries, and host fallbacks explicit rather than implicit. The result should be a system that can be measured honestly: accelerator compute, runtime overhead, and host-side work should not be conflated.

1.3 Project Scope and Approach

TinyNPU approaches this problem with four coupled components:

1. **A parameterized mixed-precision accelerator:** TinyNPU is centered around an output-stationary $N \times N$ systolic array, a unified buffer, streamer logic that feeds the array, and a post-processing unit that performs bias addition, rescaling, activation, saturation, and packed writeback.

2. **Packed integer computation:** The processing elements support packed INT16, INT8, and INT4 execution modes. Throughput therefore scales with precision reduction, not by changing the global architecture, but by packing more useful work into the same datapath.

3. **A segmented compiler/runtime:** The software flow traces supported PyTorch models, partitions them into accelerator segments and explicit host-side operations, lowers legal accelerator work into instruction and buffer images, and executes the resulting plan on either host emulation or RTL simulation.

4. **Verification and measurement infrastructure:** The project includes compiler-owned expected tensors, runtime verification points, repeated-run sessions, and benchmark accounting that separates accelerator compute from protocol and software overhead.

The current project state should be read with that scope in mind. TinyNPU is already capable of running a trained MNIST model through the full compiler/runtime stack and of benchmarking a controlled multi-tile matrix multiplication workload. At the same time, workload coverage is still narrow, and the current results are most useful for understanding architecture, packing, compiler boundaries, and runtime cost rather than for claiming broad deployment readiness.

The rest of this report follows that logic. The next section positions TinyNPU relative to relevant literature in mixed-precision quantization, matrix multiplication, and accelerator design. The system model section then explains the implemented hardware and software stack in detail, followed by the experimental method, the current results, and the most important next steps.

2 Comparative Literature Survey

2.1 Mixed-Precision Quantization

Mixed-precision quantization work is the clearest algorithmic motivation for TinyNPU. Wu et al. formulate layer-wise quantization as a search problem and show that different layers can be assigned different bit-widths without treating the whole model uniformly [7]. This is the key observation behind a variable-precision accelerator: if model layers do not all need the same precision, a fixed-precision datapath cannot exploit the full efficiency opportunity.

Quantization is also relevant as a compiler concern rather than only a hardware concern. SmoothQuant is an example of pushing difficult numerical adjustments into an offline transformation so that runtime execution becomes simpler and more hardware-friendly [8]. TinyNPU follows the same general philosophy even though it does not implement the full SmoothQuant method itself: model preparation, fused rescaling, and compiler-ready conversion are treated as software responsibilities that simplify the accelerator contract.

2.2 High-Performance Matrix Multiplication

The mathematical core of TinyNPU is still matrix-style computation, so the project is strongly informed by classical GEMM literature. Goto and van de Geijn show that high-performance matrix multiplication depends not only on arithmetic throughput, but on careful tiling and data movement through the memory hierarchy [5]. The same principle appears in TinyNPU at a smaller scale. When matrices exceed the dimensions of the systolic array, the compiler and runtime must tile along the M, N, and K dimensions while preserving the data reuse pattern expected by the array and its post-processing path.

2.3 Existing Accelerator Architectures

On the accelerator side, Google's TPU remains the canonical reference for systolic-array-style matrix acceleration [6]. Gemmini is especially relevant because it demonstrates how a systolic array, programming model, and system context can be treated as one full-stack research object rather than as an isolated RTL block [4]. That full-stack perspective is one of the strongest methodological reference points for TinyNPU.

An earlier educational TinyTPU implementation was useful as a starting point for understanding systolic-array organization, but it was not a good endpoint for this project. Extending it toward mixed precision, tiled execution, and cleaner post-processing exposed the limits of treating the datapath as the whole problem. TinyNPU therefore diverged into a custom implementation with a different software contract, explicit

host/accelerator boundaries, and a compiler/runtime that owns packing and verification rather than leaving those concerns informal.

Taken together, these references define the design space in which TinyNPU sits. It is not trying to outscale large industrial accelerators. Its niche is a compact mixed-precision accelerator system in which the hardware, packing model, compiler/runtime, and measurement methodology can all be studied together in one working platform.

3 Developed Approach and System Model

The system is easier to understand when it is described as four cooperating subsystems: the TinyNPU hardware itself, the quantizer and model-preparation path, the compiler, and the runtime. The earlier draft used a more abstract "data model / structural model / dynamic model" split, but that hides the actual engineering boundaries of the project. What matters here is not an abstract taxonomy; it is how tensors move through these four concrete parts of the stack.

3.1 System Overview

At a high level, the flow is straightforward. A model starts either as a quantized PyTorch module or as a compiler-ready artifact produced by the quantization path. The compiler traces the model, partitions it into accelerator-supported regions and explicit host operations, lowers legal regions into TinyNPU instruction and buffer images, and packages the result as a compiled artifact. The runtime then executes that artifact step by step, loading the accelerator when needed, invoking host-side operations when needed, and checking expected tensors when verification is enabled. Inside the hardware, the unified buffer, streamers, systolic array, and post-processing unit implement the actual matrix-style execution path.

That end-to-end view is important because the main design problem is not only to build a datapath. It is to maintain a clean contract between logical tensors, physical packed layouts, host-side fallback behavior, and the actual hardware pipeline. The rest of this section therefore explains the system in the same order: hardware first, then model preparation, then compilation, then runtime execution and verification.

3.2 NPU

The TinyNPU hardware is organized around a control path and a compute path. The control path is responsible for MMIO-driven loading, instruction sequencing, and control-state progression. The compute path consists of the unified buffer, streamer logic, a parameterized $N \times N$ output-stationary systolic array, and the post-processing unit. During active computation, data is read from the unified buffer, oriented by the streamers, consumed by the processing elements, and then drained through the post-processing unit before being written back in packed form.

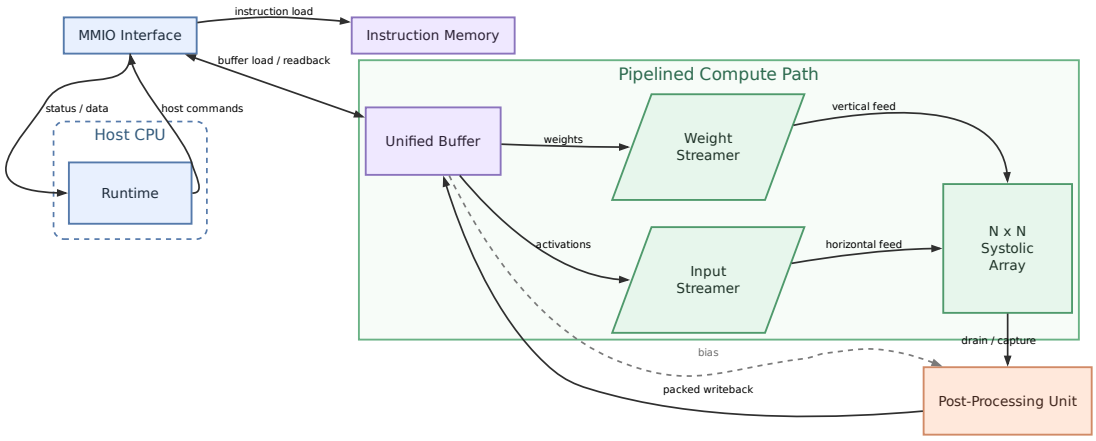

Figure 3.1: Top-level TinyNPU hardware architecture.

Figure 3.1 shows the top-level organization used throughout this report. The host-side runtime communicates with the accelerator through the MMIO interface, which is responsible for instruction loading and buffer access. Inside the accelerator, the unified buffer, input streamer, weight streamer, and systolic array form the main pipelined compute path. The post-processing unit is shown separately because it becomes active during the later drain and writeback phase rather than during the main feed phase of the array. This distinction is important for explaining both data movement and measured execution cost.

The unified buffer is not just a generic scratchpad in this design. Architecturally, it behaves as a true wide dual-port memory with a masked write path. One read side serves the activation stream, while the other serves the weight stream; occasional control-unit random reads are arbitrated onto the activation-side port. Because the buffer is wide and row-oriented, this organization is also compatible with a banked memory implementation rather than requiring a single monolithic custom memory block.

The processing element is where mixed precision becomes concrete. Each PE supports packed low-precision arithmetic rather than only a single scalar multiply. In the current implementation, a PE can perform four INT4 products, two INT8 products, or one INT16 product in a step, while accumulating the resulting partial sum in stationary form. The array therefore gains throughput both from spatial parallelism across rows and columns and from packed arithmetic within each PE.

The control path is also observable, not only functional. The controller exposes internal state counters when performance monitoring is enabled, which makes it possible to see how much time is spent in fetch, decode, feed, wait, drain, and writeback states instead of treating the whole accelerator run as a black box.

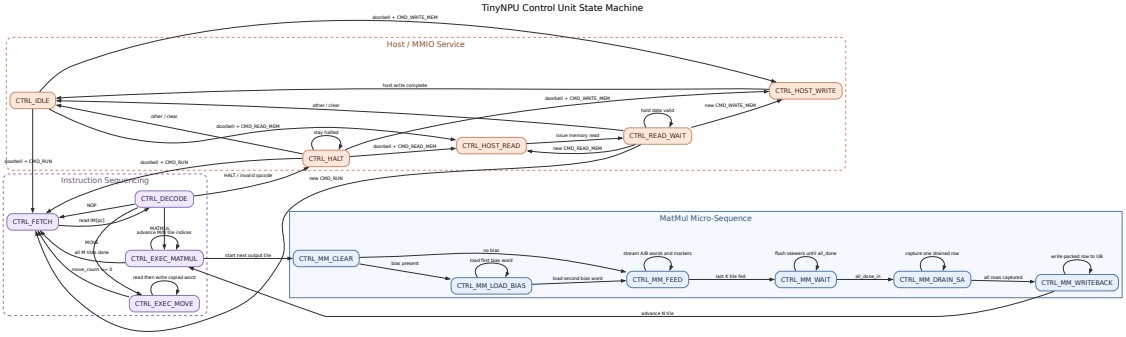

Figure 3.2: Control-unit states tracked by the internal performance counters, grouped by host service, instruction sequencing, and the matrix-multiplication micro-sequence.

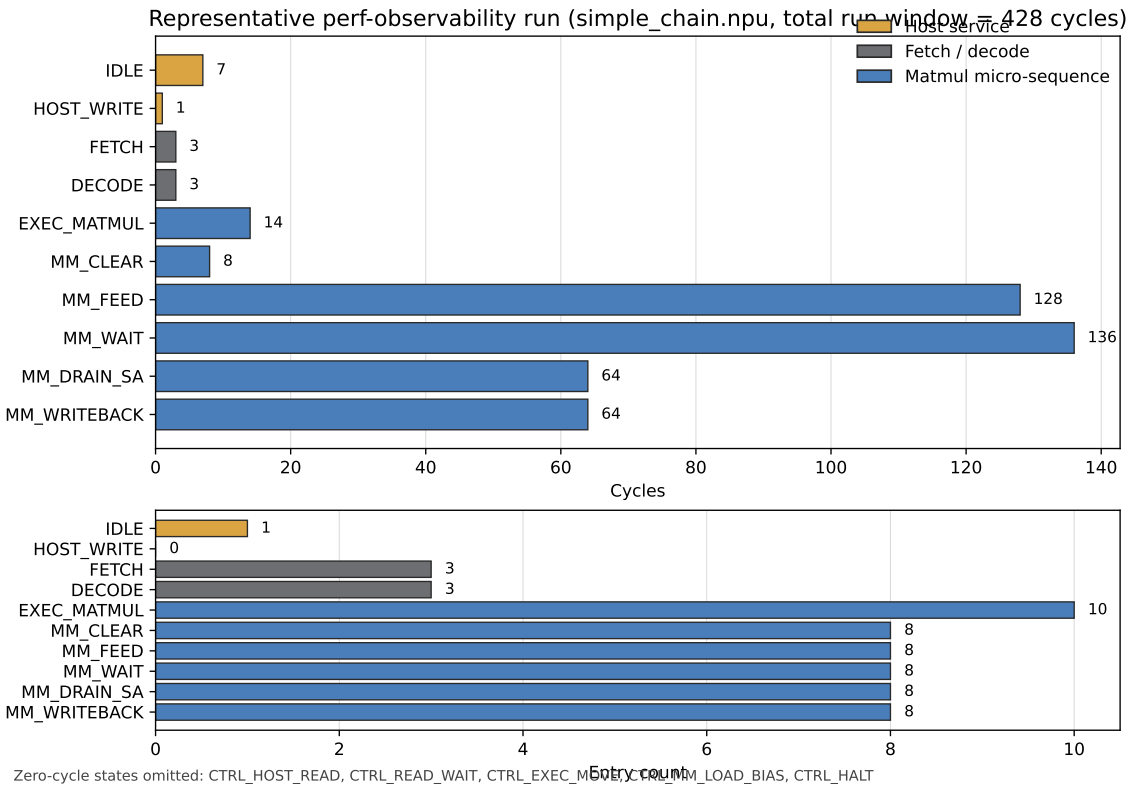

Figure 3.3: Measured per-state cycle and entry breakdown for a representative perf-observability run on `simple_chain.npu`. Zero-cycle states are omitted for readability.

The measured breakdown in Figure 3.3 makes the main control cost visible. In the representative run shown here, `CTRL_MM_FEED` and `CTRL_MM_WAIT` dominate the run window, while drain and writeback contribute a second large block of time. Fetch and decode are small by comparison. This is useful later in the performance discussion because it shows that the integration gap is not just "software overhead"; part of it is still control and synchronization work inside the accelerator run window itself.

3.3 Quantizer and Model Preparation

The quantization path exists to turn trained models into artifacts the compiler can actually lower. TinyNPU does not assume that generic floating-point PyTorch modules can be handed directly to the accelerator. The model-preparation code therefore carries explicit quantization metadata, compiler-ready conversion utilities, and calibration steps that make tensor ranges and signedness visible before compilation.

This part of the stack also includes the current mixed-precision helper. Its purpose is limited and should be stated clearly: it is not a global bit-allocation solver. Instead, it provides a practical one-layer-at-a-time sensitivity workflow that can generate usable mixed-precision configurations, recalibrate them, and pass the result onward to the compiler-ready conversion path.

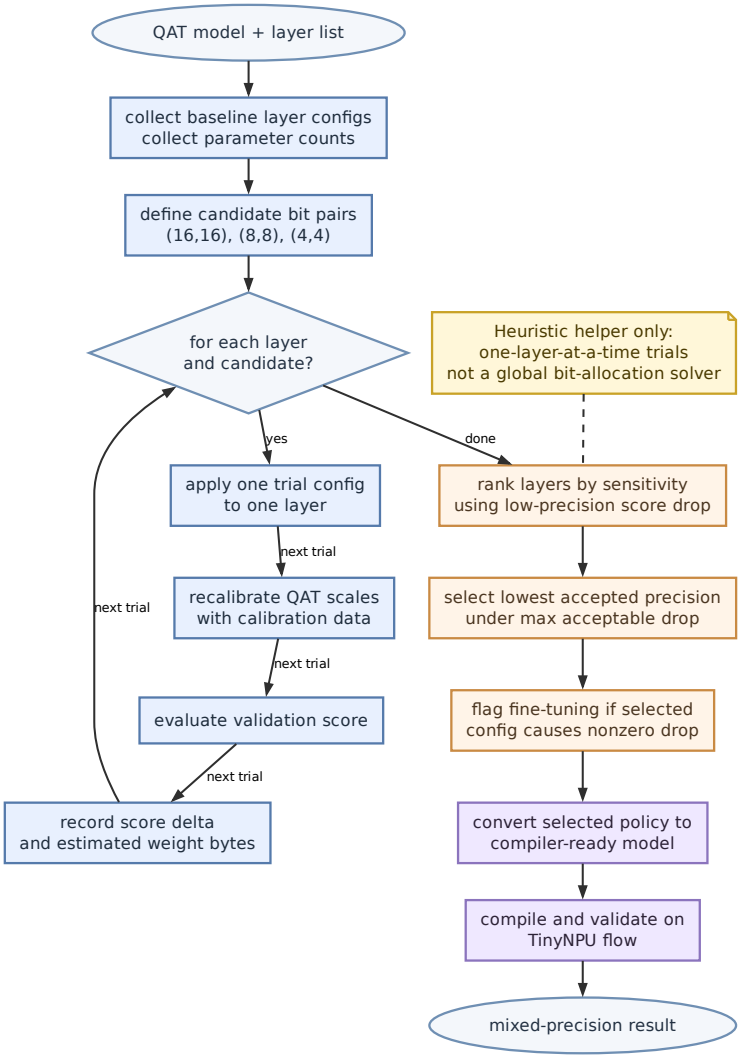

Figure 3.4: Heuristic mixed-precision selection flow used to obtain compiler-ready model variants.

The important role of this subsystem is therefore not to solve quantization research as a whole. It is to provide a disciplined bridge from trained or quantized models into forms that the TinyNPU compiler can lower without losing track of numerical meaning.

3.4 Compiler

The compiler is responsible for converting model structure into explicit execution structure. Rather than generating a monolithic accelerator-only program for the whole model, the compiler constructs an `ExecutionPlan` whose steps are typed as `NpuSegment`, `HostOp`, or `VerifyTensor`. `NpuSegment` groups accelerator-supported matrix-style work. `HostOp` keeps unsupported or intentionally host-side work explicit. `VerifyTensor` allows compiler-owned expected tensors to be checked later at runtime.

This representation is the main reason the stack remains analyzable. Unsupported behavior is not hidden, and accelerator boundaries are not implicit. The compiler can therefore make packing, readback, and verification visible rather than treating them as side effects.

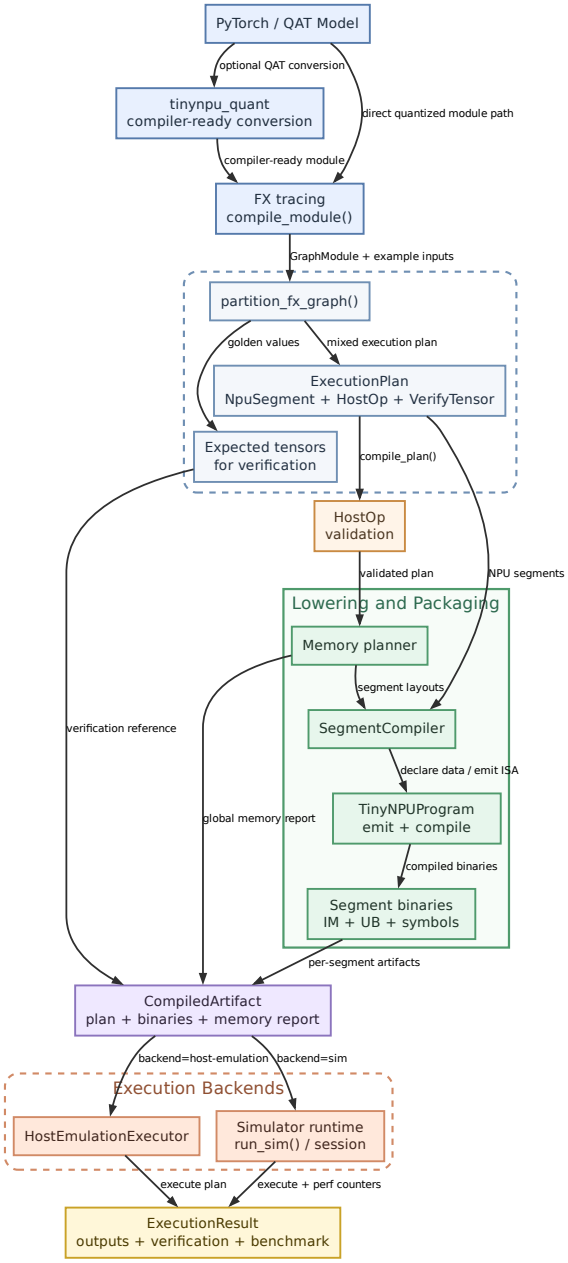

Figure 3.5: Compiler and runtime flow used by the TinyNPU JIT path.

Figure 3.5 summarizes the implemented software flow. A model enters either directly as a supported quantized PyTorch module or through a compiler-ready conversion path built around `tinynpu_quant`. The JIT frontend traces the model with FX, partitions it into a mixed execution plan, validates host operations, and lowers each accelerator segment through the memory planner and `SegmentCompiler`. The resulting `CompiledArtifact` contains the execution plan, per-segment IM/UB binaries, expected tensors for verification, and the memory-layout information required by the runtime.

Packing is part of the compiler contract as well. The backend uses role-specific phys-

ical layouts for activation-side tensors, weight-side tensors, output buffers, and bias payloads. These are physical storage forms rather than semantic tensor meanings, but they are essential because legal execution depends on matching memory layout to the real feed directions and writeback format of the NPU.

3.5 Runtime and Verification

The runtime executes the compiled artifact as a mixed host-accelerator loop. For each step in the execution plan, it either launches an accelerator segment, executes an explicit host operation, or checks a verification tensor. The key point is that host-side work is never hidden. If a step remains on the host, it remains visible in the plan and in the benchmark accounting.

The runtime also owns repeated-run behavior. It can preserve reusable state such as static unified-buffer contents and preloaded instruction-memory images across runs. This does not remove all setup work, but it makes repeated inference more realistic and exposes a measurable optimization path for reducing avoidable overhead.

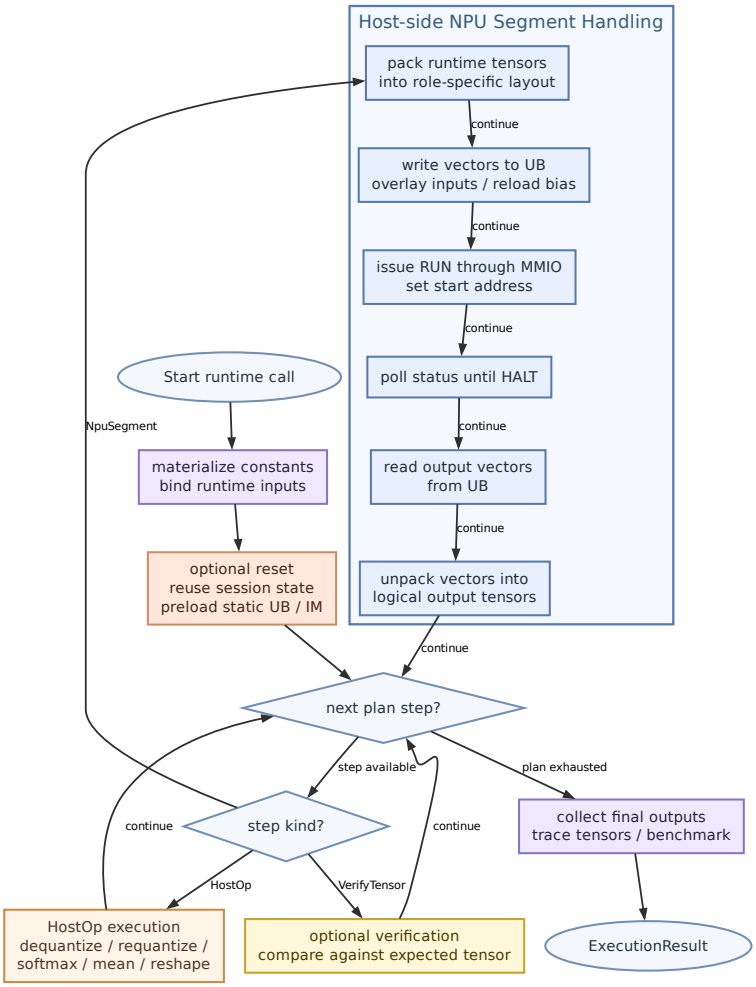

Figure 3.6: Host-side runtime flow for executing a compiled TinyNPU artifact.

Figure 3.6 shows the same loop from the host-side orchestration perspective. For accelerator segments, the runtime first packs logical tensors into the physical layouts expected by the datapath, writes the necessary vectors into the unified buffer, launches execution through MMIO, waits for completion, and then reads and unpacks the resulting vectors back into logical tensors. Host-side operations such as requantization, dequantization, softmax, mean, and shape movement appear explicitly in the same loop rather than being hidden behind the accelerator interface.

Verification is built into this same path. Compiler-owned expected tensors can be checked during execution, which means correctness is not limited to end-of-run output comparison. Benchmarking is also integrated here: the runtime can record hard RTL cycle counts for accelerator run windows while separately accounting for software-side overhead such as packing, MMIO traffic, and readback.

3.6 Current Scope Boundaries

The current system is deliberately narrower than a general-purpose deployment stack. The mixed-precision helper is meant to produce usable compiler-ready configurations, not to solve the global bit-allocation problem. The compiler handles a bounded set of operators and makes unsupported behavior explicit through host operations rather than pretending to provide transparent full-model lowering. Workload coverage is also still narrow: the current report is grounded in one end-to-end model case and one controlled kernel benchmark. These limitations are acceptable for the present project scope because the main objective is to build and understand a working accelerator system, not to claim a finished production compiler.

4 Experimental Setup and Design

The project was evaluated primarily in simulation, using both host-side execution and RTL-backed cocotb tests. This was necessary because the main goal of the project is not only to confirm functionality, but also to understand how compiler decisions, packing format, and runtime policy affect accelerator behavior before any physical implementation is assumed. Host emulation and RTL simulation share the same execution-plan concept, which made it possible to compare compiler behavior against a golden software model while still collecting hardware-facing evidence.

The experimentation setup contains three layers. First, unit and regression tests validate compiler transformations, execution-plan behavior, memory-planning behavior, and host-op semantics. Second, RTL simulation validates full hardware/software interaction and allows performance counters to be read during accelerator execution. Third, a reproducible Yosys-based synthesis flow provides rough logic and resource estimates for the current top-level design both with and without treating the major memories as abstract macros.

The RTL performance-monitoring path is particularly important for interpreting the accelerator measurements. With performance monitoring enabled, the control unit reports not only a total run-window cycle count, but also per-state cycle and entry counts for its internal control states. This makes it possible to distinguish broad classes of activity such as host service, fetch/decode, feed, wait, drain, and writeback when analyzing where the accelerator is actually spending time during a segment.

The current empirical evidence is intentionally limited and should be described honestly in the report. There is one full end-to-end model case study: a trained MNIST flow that passes through model preparation, compiler-ready conversion, segmented compilation, and RTL-backed validation. There is also one controlled performance case study: a multi-tile matrix multiplication workload large enough to force tiling on the current array configuration. This benchmark is not intended to represent full-model latency by itself; it is a kernel-level workload chosen to isolate the accelerator's core behavior and the effect of integration overhead.

The performance methodology separates different cost categories on purpose. Accelerator compute cycles are taken from RTL performance counters during the run window of a segment. Software-side overhead such as loading buffer images, issuing control transactions, packing and unpacking data, and executing host-only operations is accounted for separately. Several CPU reference models are also applied on the software side, but these are analytic comparison models rather than measurements from a physical reference processor. The purpose of those models is to bound and interpret the accelerator's behavior, not to claim a definitive CPU-versus-NPU benchmark.

5 Comparative Evaluation and Discussion

The current evaluation supports several concrete conclusions, but it should be read with the correct scope. The evidence is sufficient to show that the stack works end to end and that the accelerator core is meaningful. It is not yet broad enough to support sweeping claims about general workload coverage.

First, the end-to-end software and hardware path is real. A trained MNIST model has been prepared for the compiler, compiled through the segmented JIT flow, and validated against RTL. This is important because the central claim of the project is not simply that individual kernels run, but that a model can move from PyTorch-side preparation into a real TinyNPU execution artifact and still preserve host/RTL correctness.

Second, the performance tooling reveals a clear separation between core compute strength and integration cost. On the current multi-tile matrix multiplication benchmark, the RTL run window for the accelerator is only 369 cycles. Depending on the chosen CPU reference model, this translates to a large pure compute speedup range, but the integration-adjusted speedup is much lower because runtime and protocol overhead still matter. This is exactly the kind of conclusion the benchmark path was designed to support: the datapath itself may be strong, while the surrounding software and interface contract still leave room for improvement.

Table 5.1: Controlled multi-tile matrix multiplication benchmark results for the 13×17 by 17×19 workload.

Reference model	CPU replaced cycles	Pure acceleration speedup	Integration-adjusted speedup
Unpipelined scalar	51870	140.57×	8.39×
Ideal issue-1	30628	83.00×	5.20×
Five-stage in-order	43472	117.81×	7.09×

For the same workload, the measured accelerator run window was 369 cycles. Table 5.1 makes the key point visible: the compute core itself is strong, but integration-adjusted speedup is much lower than pure compute speedup because runtime and interface costs are still substantial.

Third, repeated-run runtime work is already justified by measured behavior. On a small repeated-run benchmark, static persistence reduced the modeled NPU-side overhead from 3036 to 1744 units, which corresponds to roughly a 42.6% reduction. On trained MNIST, where the workload is more activation-heavy, the same idea still reduced repeated-run unified-buffer traffic from 17608 words to 14920 words, which is about a 15.3% reduction. These numbers do not claim that the whole problem is solved; they show that repeated-run state reuse is a real and measurable optimization direction.

Table 5.2: Observed effect of repeated-run persistence on two workloads.

Workload	**Metric**	**Baseline**	**Persistence path**	**Observed effect**
Small repeated-run artifact	NPU overhead cycles	3036	1744	42.6% lower overhead
Trained MNIST	UB words per run	17608	14920	15.3% lower steady-state traffic

Fourth, the synthesis-oriented characterization confirms that the design is no longer trivial in hardware terms. Under the reproducible Yosys flow, the top-level design synthesized to an estimated 25117 logic cells when the major memories were abstracted and 33644 logic cells when the current memory implementation was kept, with 227 DSP48 blocks in both cases. These numbers are only rough synthesis proxies, not final silicon area, but they are enough to show that TinyNPU is a substantial compute block whose area and timing behavior now deserve serious attention.

The most important interpretation of these results is that the project's center of gravity has shifted. The compiler and runtime exist well enough to run real workloads. The pressing questions are now about workload fit, cycle reduction, memory sizing, and hardware cost, not about whether a compiler path can exist at all.

6 Conclusion and Future Work

TinyNPU has reached the point where it should be treated as a real accelerator platform rather than as a collection of disconnected experiments. The project now contains a usable hardware architecture, a PyTorch-oriented model preparation flow, a segmented compiler/runtime, explicit host-boundary handling, repeated-inference support, verification infrastructure, and benchmark tooling capable of separating compute from overhead. That is a substantial transition from the early stage of simply proving that isolated RTL blocks or simple kernels behaved correctly.

At the same time, the project makes its own limitations visible in a useful way. The current system still pays a noticeable cost for segmentation, state movement, packing, and host-visible boundaries. The mixed-precision path is practical for preparing compiler-ready models, but it is not intended to solve the global bit-allocation problem. Workload coverage is still narrow, and the current performance story is based on one end-to-end model and one controlled kernel benchmark rather than on a broad suite of models.

Those limitations point directly to the next phase of work. The highest-value future work is no longer "make the compiler exist." It is to bring more models onto the stack, reduce avoidable runtime and interface overhead, increase the size of useful accelerator regions, and use real workloads to justify changes in unified-buffer and instruction-memory sizing. On the hardware side, synthesis and timing characterization should be extended beyond rough proxy numbers. On the software side, additional compiler work should be driven by measured workload needs rather than by abstraction for its own sake. That is the most credible path forward for TinyNPU as a research system.

A Benchmark Cost Model

The benchmark path used in this project combines measured RTL cycle counts with an analytic reference-CPU model. The goal is not to claim a definitive comparison against a specific commercial processor. Instead, it is to make the accelerator results interpretable by separating three different questions:

1. How much logical work would a scalar CPU have performed for the same operation?
2. How many cycles did the accelerator actually spend in its measured run window?
3. How much extra cost was introduced by packing, MMIO traffic, and host-side orchestration?

To support that split, benchmark entries are grouped into four buckets:

1. **`cpu_replaced`:** logical work that the CPU would have performed instead of an accelerator segment. For matrix multiplication, this includes scalar reads,

multiply-accumulate work, loop-control branches, optional bias addition, requantization, and output writes.

2. **`npu_compute`:** measured RTL cycle counts collected during the accelerator run window with performance monitoring enabled.

3. **`npu_overhead`:** software-visible and protocol-visible work required to use the accelerator, such as packing, interface writes, interface reads, instruction loading, and readback.

4. **`host_intrinsic`:** work that intentionally remains on the host side, such as host operations that are not lowered into accelerator segments.

Primitive work is first counted in an abstract instruction-class form. The counted fields include reads, writes, adds, multiplies, branches, shifts, clamps, bitwise operations, divides, nonlinear helper calls, interface writes, and interface reads.

An analytic CPU model is then applied to those counts:

$$\texttt{cycles} = \sum_i \texttt{count}_i \times \texttt{cost}_i$$

The three reference models currently used in the project are shown in Table A.1. These are implemented directly in the benchmark code and reuse the same counted trace; only the CPI assumptions change.

Table A.1: Reference CPU cost models used in the TinyNPU benchmark flow.

Event class	**Unpipelined scalar**	**Ideal issue-1**	**Five-stage in-order**
Reads / writes	2 / 2	1 / 1	1 / 1
Adds / shifts / clamps / bitops	1	1	1
Integer multiply	4	1	3
Branch	1	1	2
Divide	16	1	16
Nonlinear helper	8	1	8
Interface read / write	1 / 1	1 / 1	1 / 1

The reported speedups are derived from those bucket totals:

$$S_{\text{pure}} = \frac{C_{\text{cpu}}}{C_{\text{npu}}}$$
$$S_{\text{int}} = \frac{C_{\text{cpu}}}{C_{\text{npu}} + C_{\text{ovh}}}$$

Here:

- C_{cpu} denotes the modeled `cpu_replaced_cycles`.

- C_{npu} denotes the measured `npu_compute_cycles`.
- C_{ovh} denotes the counted `npu_overhead_cycles`.

This definition is why the integration-adjusted speedup is always lower than the pure accelerator speedup. The denominator includes the cost of using the accelerator, not only the measured run window inside the hardware. That is an intentional design choice in the methodology: the project is trying to understand the full cost of practical use, not only the inner datapath in isolation.

References

[1] Georgios Armeniakos, Francesco Maria Ottavi, Angelo Garofalo, and Luca Benini. Mixed-precision support for RISC-V soft-SIMD extensions, 2024.

[2] Quim Fornt, Georgios Armeniakos, Pasquale Davide Schiavone, and Luca Benini. Mix-GEMM: Mixed-precision RISC-V matrix multiplication using binary segmentation. *IEEE Transactions on Computers*, 2025.

[3] Angelo Garofalo, Francesco Conti, Giuseppe Tagliavini, Davide Rossi, and Luca Benini. XpulpNN: Enabling energy efficient and flexible inference of quantized neural networks on RISC-V based iot end nodes. *IEEE Transactions on Emerging Topics in Computing*, 10(4):1849–1863, 2021.

[4] Hasan Genc, Ameer Haj-Ali, Vivek Ittycheriah, et al. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration. *Proceedings of the 58th Annual Design Automation Conference (DAC)*, pages 769–774, 2021.

[5] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):12:1–12:25, 2008.

[6] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.

[7] Bichen Wu, Yanghan Wang, Peizhao Zhang, Yonglong Tian, Peter Vajda, and Kurt Keutzer. Mixed precision quantization of convnets via differentiable neural architecture search, 2018.

[8] Guangxuan Xiao, Ji Lin, Meryem Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 38087–38099, 2023.